

NHCC Enterprise Management Portal & Finara AI: System Context

Note for Claude / AI Assistant: This document contains the complete architectural, technical, and feature-level context of the NHCC application. Use this document as the absolute source of truth when answering prompts or generating new code for this project.

1. Project Overview

This is a monorepo application containing two primary modules:

1. **NHCC Portal:** An enterprise project management system for the National Housing and Construction Company. It handles construction projects, tasks, budgets, messaging, approvals, and employee roles.
2. **Finara AI:** A built-in personal finance advisor module for employees, featuring advanced machine learning for budget categorization and voice-assisted navigation.

2. Technology Stack

- **Frontend:** React.js (built with Vite).
- **Frontend Styling:** Vanilla CSS and TailwindCSS. The UI relies heavily on modern **3D Neumorphism and Glassmorphism** aesthetics.
- **Backend:** Node.js with Express.
- **Database:** SQLite.
- **ORM:** Prisma (using strict camelCase in schema, but mapping to snake_case in the DB).
- **Real-Time Engine:** Socket.io (for live data syncing across clients).
- **Authentication:** JWT (JSON Web Tokens) with `bcryptjs` for password hashing.

3. Core Enterprise Features

A. Role-Based Access Control (RBAC) & Row-Level Security (RLS)

- **RBAC:** The frontend (`client/src/layouts/NHCCLayout.jsx`) dynamically hides/shows sidebar navigation items based on the user's role (e.g., CEO, ICT, CHIEF_ENGINEER, PROJECT_IMPLEMENTER).
- **RLS:** The backend enforces Row-Level Security via a central middleware filter in `server/routes/crud.js`. If a `PROJECT_IMPLEMENTER` queries tasks, Prisma dynamically injects a `WHERE assignee_id = {userId}` clause so they can only fetch their own data.

B. Prisma Audit Trails

- The backend utilizes Prisma `$extends` and `AsyncLocalStorage` to intercept every `CREATE`, `UPDATE`, and `DELETE` operation.
- Every mutation automatically generates an immutable record in the `AuditLog` table containing the user ID, timestamp, IP address, and JSON diffs of the old vs. new values.

C. Real-Time Concurrency (WebSockets)

- When any CRUD operation occurs via the REST API, the backend emits a `Socket.io` broadcast (`entity:updated`, `entity:created`).
- The React frontend listens for these events and optimistically updates its local UI state without requiring a browser refresh.

4. Machine Learning & AI Integrations

The application features three distinct, locally executed Machine Learning algorithms:

1. Monte Carlo Simulation (Project Risk Forecasting):

- Located in: `server/routes/ml.js` (Endpoint: `/api/ml/forecast/budget`).
- Functionality: Runs 1,000 parallel simulation iterations of the remaining project timeline. It injects random volatility multipliers (0.7x to 1.6x) to calculate a probabilistic statistical confidence interval (e.g., 90% probability of finishing under budget) rather than using simple division.

2. Naive Bayes Probabilistic Classifier:

- Located in: `client/src/lib/ai-engine.js`.
- Functionality: Replaced basic keyword matching. It uses a Naive Bayes algorithm with Laplace smoothing to calculate the mathematical log-likelihood of a text description belonging to a specific financial expense category.

3. Cosine Similarity NLP (Natural Language Processing):

- Located in: `client/src/lib/ai-engine.js` (Voice Navigation).
- Functionality: Transcribed voice commands are tokenized into Term Frequency (TF) vectors. The engine computes the Cosine Similarity angle between the user's input vector and predefined intent vectors to accurately route commands even if the phrasing is unpredictable.

5. Seed Data & Database Context

- Seed script: `server/db/init.js`.
- The database is prepopulated with Ugandan + English names for authenticity (e.g., Fred Mukisa as CEO, Jane Namubiru as Chief Engineer, Michael Kintu for Finance).
- Demo passwords are all `password123`.

6. Directory Structure (High Level)

```
NHCC_APP/
├── client/                                # Vite + React Frontend
│   ├── src/
│   │   ├── components/                  # Reusable UI components (Neumorphic design)
│   │   ├── contexts/                   # React Context (AuthContext, ThemeContext)
│   │   ├── layouts/                     # NHCCLayout.jsx (RBAC logic)
│   │   ├── lib/                         # ai-engine.js (NLP & Naive Bayes)
│   │   ├── pages/                       # Route views
│   │   └── index.css                    # Core CSS (Glassmorphism tokens)
│   └── vite.config.js                   # Configured to proxy /api to localhost:3001
├── server/                              # Node.js + Express Backend
│   ├── db/                             # SQLite file (app.db) and init.js
│   ├── lib/                             # prisma.js (Audit extension) & async-context.js
│   ├── middleware/                      # auth.js (JWT validation)
│   ├── prisma/                          # schema.prisma
│   └── routes/                          # crud.js (Dynamic Prisma routing & RLS), ml.js
```

7. How to run

- **Backend:** `cd server && npm run start` (Runs on port 3001)
- **Frontend:** `cd client && npm run dev` (Runs on port 5173)

End of Context Document.